

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.

Constraints:

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

Pseudocode:

```
BEGIN FaceDetectionAndRecognitionSystem

// ----- 1. INITIALIZATION -----
LOAD face_detector_model      // e.g., Haar Cascade / DNN face detector
LOAD face_recognizer_model    // e.g., LBPH / trained embedding model
LOAD registered_faces_dataset // known face encodings + labels
SET frame_skip_counter = 0
SET frame_skip_threshold = N  // process every Nth frame if needed

OPEN video_capture FROM camera_source
IF video_capture NOT opened THEN
    PRINT "Error: Cannot access camera"
    EXIT
END IF

// ----- 2. MAIN REAL-TIME LOOP -----
WHILE camera_is_streaming DO

    // --- Image Acquisition ---
```

```

SUCCESS, frame = video_capture.READ_FRAME()
IF NOT SUCCESS THEN
    PRINT "Frame capture failed"
    BREAK
END IF

frame_skip_counter = frame_skip_counter + 1
IF frame_skip_counter MOD frame_skip_threshold != 0 THEN
    DISPLAY frame
    CONTINUE      // skip heavy processing to save time
END IF

// --- Preprocessing ---
gray_frame = CONVERT_TO_GRAYSCALE(frame)
gray_frame = APPLY_HISTOGRAM_EQUALIZATION(gray_frame) // normalize lighting
resized_frame = RESIZE(gray_frame, scale_factor)      // speed up detection

// --- Face Detection (multiple faces) ---
detected_faces = face_detector_model.DETECT_MULTI_SCALE(
    resized_frame,
    scaleFactor = 1.1,
    minNeighbors = 5,
    minSize = (30, 30))

face_count = LENGTH(detected_faces)

// --- Process Each Detected Face ---
FOR EACH face_rect IN detected_faces DO

    (x, y, w, h) = face_rect
    face_roi = EXTRACT_REGION(gray_frame, x, y, w, h)
    face_roi = RESIZE(face_roi, standard_size)      // e.g., 200x200

    // --- Feature Extraction ---
    face_embedding = EXTRACT_FEATURES(face_roi)      // LBPH histogram / CNN embedding

    // --- Recognition ---
    best_match_label = "Unknown"

```

```

best_match_score = INFINITY

FOR EACH (known_embedding, known_label) IN registered_faces_dataset DO
    distance = COMPUTE_DISTANCE(face_embedding, known_embedding)
    IF distance < best_match_score THEN
        best_match_score = distance
        best_match_label = known_label
    END IF
END FOR

IF best_match_score > recognition_threshold THEN
    best_match_label = "Unknown"
END IF

// --- Visualization ---
DRAW_RECTANGLE(frame, x, y, w, h, color = GREEN, thickness = 2)
DRAW_LABEL(frame, best_match_label, position = (x, y - 10))

// Optional: log recognized identity with timestamp
IF best_match_label != "Unknown" THEN
    LOG_ACCESS_EVENT(best_match_label, CURRENT_TIMESTAMP())
END IF

END FOR

DRAW_TEXT(frame, "Faces Detected: " + face_count, position = (10, 30))

// --- Display Output ---
DISPLAY frame IN window "Real-Time Face Recognition"

// --- Efficient Loop Control ---
key = WAIT_KEY(delay = 1) // minimal delay to keep loop responsive
IF key == 'q' THEN
    BREAK // graceful exit condition
END IF

END WHILE

```

```
// ----- 3. CLEANUP -----  
RELEASE video_capture  
CLOSE all display windows  
  
END FaceDetectionAndRecognitionSystem
```

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.
- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.

Constraints:

- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

Pseudocode:

```
BEGIN VehicleDetectionTrackingCountingSystem  
  
// ----- 1. INITIALIZATION -----  
LOAD vehicle_detector_model    // e.g., YOLO / Haar / background subtractor  
CREATE tracker_registry = EMPTY MAP // maps tracking_id -> tracker object & centroid history  
SET next_tracking_id = 0  
SET vehicle_count = 0  
DEFINE region_of_interest (ROI)    // polygon or bounding area in frame  
DEFINE counting_line (point_A, point_B) // virtual line for crossing detection  
CREATE counted_ids_set = EMPTY SET // prevents double counting  
  
OPEN video_capture FROM traffic_camera_source
```

```

IF video_capture NOT opened THEN
    PRINT "Error: Cannot access video stream"
    EXIT
END IF

// ----- 2. MAIN REAL-TIME LOOP -----
WHILE video_stream_active DO

    // --- Video Acquisition ---
    SUCCESS, frame = video_capture.READ_FRAME()
    IF NOT SUCCESS THEN
        PRINT "End of stream or read failure"
        BREAK
    END IF

    // --- Preprocessing ---
    roi_frame = MASK_TO_REGION_OF_INTEREST(frame, ROI)
    gray_frame = CONVERT_TO_GRAYSCALE(roi_frame)
    blurred_frame = APPLY_GAUSSIAN_BLUR(gray_frame, kernel_size)
    foreground_mask = BACKGROUND_SUBTRACTOR.APPLY(blurred_frame)
    foreground_mask = APPLY_MORPHOLOGICAL_OPS(foreground_mask) // erode/dilate to remove noise

    // --- Vehicle Detection ---
    detections = vehicle_detector_model.DETECT(roi_frame)
    // each detection = (bounding_box, confidence, class_label)
    detections = FILTER_BY_CONFIDENCE(detections, min_confidence)
    detections = APPLY_NON_MAX_SUPPRESSION(detections)

    // --- Multi-Object Tracking ---
    current_centroids = COMPUTE_CENTROIDS(detections)
    matched_pairs, unmatched_detections, unmatched_tracks =
        ASSOCIATE_DETECTIONS_TO_TRACKS(current_centroids, tracker_registry)
    // e.g., via IoU matching or centroid distance + Kalman filter prediction

    // Update matched tracks
    FOR EACH (track_id, detection) IN matched_pairs DO
        UPDATE_TRACKER(tracker_registry[track_id], detection)
        APPEND detection.centroid TO tracker_registry[track_id].centroid_history
    
```

```
END FOR
```

```
// Create new trackers for unmatched detections (new vehicles entering ROI)
```

```
FOR EACH detection IN unmatched_detections DO
```

```
    tracker_registry[next_tracking_id] = CREATE_NEW_TRACKER(detection)
```

```
    next_tracking_id = next_tracking_id + 1
```

```
END FOR
```

```
// Remove stale tracks (vehicle left frame / lost for too many frames)
```

```
FOR EACH track_id IN unmatched_tracks DO
```

```
    tracker_registry[track_id].missed_frames += 1
```

```
    IF tracker_registry[track_id].missed_frames > max_missed_frames THEN
```

```
        REMOVE tracker_registry[track_id]
```

```
    END IF
```

```
END FOR
```

```
// --- Counting Logic (avoid double counting) ---
```

```
FOR EACH (track_id, tracker) IN tracker_registry DO
```

```
    prev_centroid = tracker.centroid_history[LAST_INDEX - 1]
```

```
    curr_centroid = tracker.centroid_history[LAST_INDEX]
```

```
    IF prev_centroid EXISTS AND
```

```
        HAS_CROSSED_LINE(prev_centroid, curr_centroid, point_A, point_B) AND
```

```
        track_id NOT IN counted_ids_set THEN
```

```
        vehicle_count = vehicle_count + 1
```

```
        ADD track_id TO counted_ids_set // mark as counted, prevents recount
```

```
    END IF
```

```
END FOR
```

```
// --- Visualization ---
```

```
DRAW_LINE(frame, point_A, point_B, color = RED, thickness = 2)
```

```
FOR EACH (track_id, tracker) IN tracker_registry DO
```

```
    DRAW_RECTANGLE(frame, tracker.bounding_box, color = GREEN)
```

```
    DRAW_LABEL(frame, "ID: " + track_id, position = tracker.bounding_box.top_left)
```

```
END FOR
```

```
DRAW_TEXT(frame, "Vehicle Count: " + vehicle_count, position = (10, 30))
```

```
DISPLAY frame IN window "Vehicle Detection, Tracking & Counting"
```

```
// --- Efficient Loop Termination ---
```

```
key = WAIT_KEY(delay = 1)
```

```
IF key == 'q' THEN
```

```
    BREAK
```

```
END IF
```

```
END WHILE
```

```
// ----- 3. CLEANUP -----
```

```
RELEASE video_capture
```

```
CLOSE all display windows
```

```
PRINT "Final Vehicle Count: " + vehicle_count
```

```
END VehicleDetectionTrackingCountingSystem
```